

Lab 2: Heisrapport

Eirik Stokker Aksdal, Vikingur Sigurðsson

Gruppe 11

16. mars 2026

Innledning

Denne rapporten dokumenterer arbeidet med heisprosjektet tilhørende lab 2 i TTK4235 våren 2026. Prosjektets formål har vært å utvikle et styresystem i C for en fysisk heismodell på sanntidslaben, med fokus på strukturert programvareutvikling, modulær oppbygning og etterprøvable funksjonalitet i henhold til gitte kravspesifikasjoner.

I rapporten skal vi gå gjennom fremgangsmåten for utviklingen av programmet med utgangspunkt i V-modellen, samt programstruktur ved bruk av UML og til slutt en refleksjon av arbeidet rundt V-modellen og bruken av UML

Fremgangsmåte

Vi startet med en prototypefase der vi skrev noenlunde fungerende kode for å forstå det utleverte rammeverket og driveren til heisen. Videre brukte vi V-modellen og erfaringene ifra prototypingen for å strukturere, planlegge og implementere den faktiske produksjonskoden.

Fra prototypefasen fant vi blandt annet at den utleverte driverkoden var ekstremt treg og til tider gjorde at heisen kjørte forbi en etasje før den registrerte den. Derfor implementerte vi Utskrift 1 i `elevio_init` funksjonen til `elevio.c` for å forbedre responsen på driveren.

```
int flag = 1;
setsockopt(
    sockfd,
    IPPROTO_TCP,
    TCP_NODELAY,
    &flag,
    sizeof(flag)
);
```

Utskrift 1: Setter et flag i initialisering som gjør at TCP sender packets individuelt istedenfor å vente på flere packets.

Siden driveren kommuniserer med elevator serveren gjennom mange små packets over TCP så ville TCP automatisk vente på flere packets

før den sender dem som en stor packet. Til vanlig er dette fornuftig siden packetsene vi sendte var ofte mindre enn overheadet som trengtes til TCP.

Etter prototypingen begynte vi med den pragmatiske V-modellen og per den så starter vi med kravspesifikasjonene til heisen der vi tar utgangspunkt i FAT-kravene gitt i oppgavebeskrivelsen og use-case diagrammet i Figur 1.

Videre gjorde vi idemyldring for arkitekturdesign gjennom å vurdere kodene skrevet i prototypefasen der vi til slutt kom fram til hvordan bestillingssystemet og arkitekturen kunne mest fornuftig implementeres. Det var feks her at vi kom fram til at vi skulle bruke en tilstandsmaskin for sluttimplementasjonen og hvilke tilstander vi kom til å trenge som er illustrert i Figur 2.

Gitt tilstandsmaskinen kan use-case diagrammet beskrives slikt:

Velg Etasje

Pre-conditions:

Heisen er initialisert og stoppknappen er sluppet.

Trigger:

Bestillingsknapp inne i heisen blir trykket.

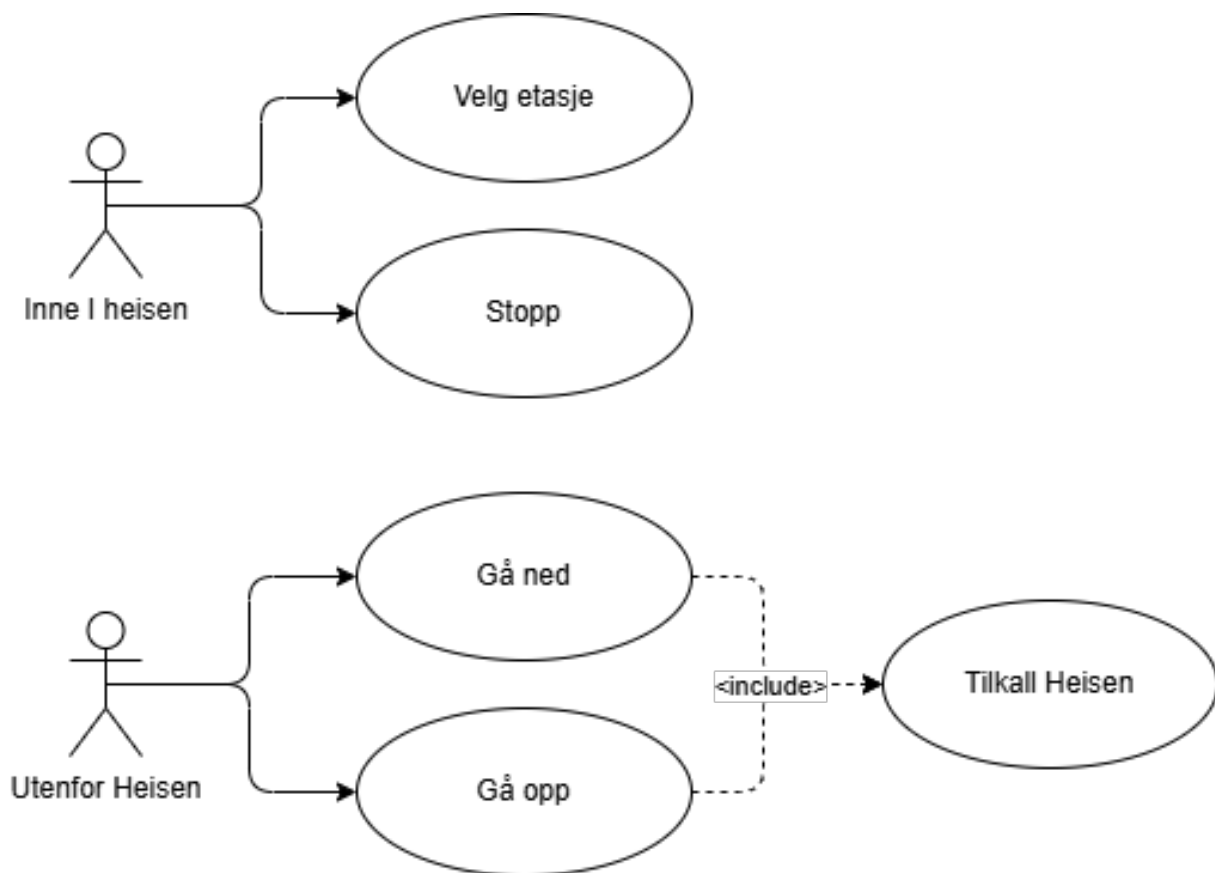
Suksess-scenario:

1. Heisens tilstand og motor settes til tilhørende retting.
2. Heisen ankommer etasjen.
3. Heisen stopper.
4. Heisen åpner døra i 3 sekunder.

Utvidelser:

1a. Hvis stoppknappen trykkes følger suksess-scenario alternativ flow ifølge use-case til stoppknappen.

2a. Døra forblir åpen dersom det er en obstruksjon



Figur 1: Use-case diagram

Stopp

Pre-conditions:

Heisen er initialisert.

Trigger:

Stoppknappen trykkes.

Suksess-scenario:

1. Heisen stopper
2. Alle bestillinger slettes.
3. Døra åpnes dersom heisen er i en etasje

Utvidelser:

- 1a. Døra forblir åpen intill 3 sekunder etter stoppknappen trykkes
- 2a. Døra forblir åpen dersom det er en obstruksjon.

Gå opp/ned

Pre-conditions:

som velg etasje

Trigger:

Knapp opp eller ned utenfor heisen blir trykket

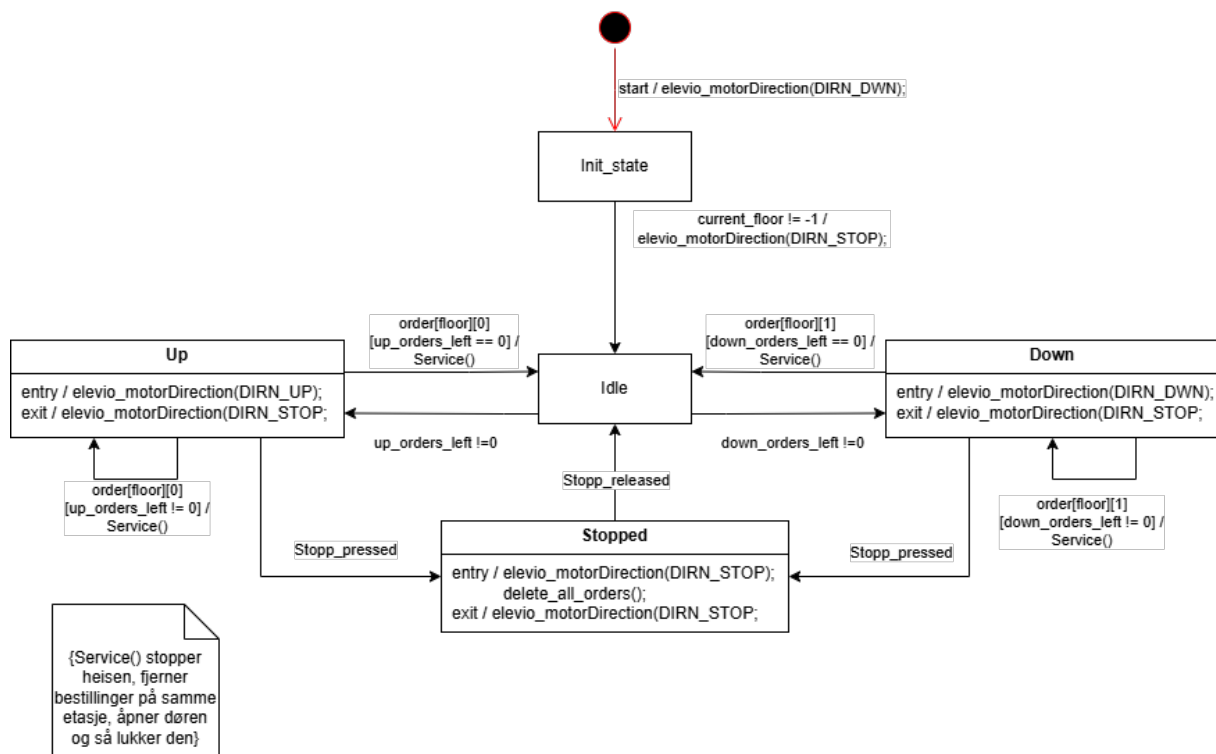
Suksess-scenario:

som velg etasje

Utvidelser:

som velg etasje

Når det gjelder heisens oppførsel bestemte vi at hvis den beveger seg i en rettning skal den fullføre alle ordre som er forran den i den retningen, med prioritet av ordre i samme rettning (dvs at den ignorerer ordre i motsatt rettning hvis ordre i samme retning er tilgjengelig forran den). Og at den går i idle dersom den ikke har flere ordre i den retningen, og at idle tilstand alltid ser etter nye ordre og går i retningen til den første ordren den registrerer.



Figur 2: Førsteutkast av tilstands diagram

Med utgangspunkt i det oppsatte rammeverket vårt, oversatte vi og implementerte vi koden vår i C.

Modulene vi endte opp med å bruke var `main.c` og `elevatorutils.c` der `main.c` inneholder tilstandsmaskinen og `elevatorutils.c` inkluderer alle hjelpefunksjoner. Som for eksempel Utskrift 5 som sjekker om det er ordre i etasjene den er i, og fullfører den dersom den er gyldig, gitt kravene.

Førsteutkastet av denne implementasjonen ga en nogenlunde fungerende kode. Hver implementerte modul ble testet og fungerte. Men den passerte ikke alle kravene til FAT testen. Spesifikt test S6 der Stopp tilstanden kunne huske dens forrige etasje, men på grunn av tilstands oppsettet, er den også avhengig av hvilken retning tilstanden er i. Derfor endte vi opp med å legge til en til tilstand som beskriver når heisen er stoppet imellom to etasjer.

Struktur

Testing og opprydding av kode slik at den fungerer bedre og ser finere ut, i henhold til V-

modellens prosess, ga det ferdige produktet beskrevet her.

Et overblikk over programmet funksjon kan gis med det ferdige tilstandsdiagrammet i Figur 3 og kalssedigrammet i Figur 4 Programets funksjon kan beskrives som følger.

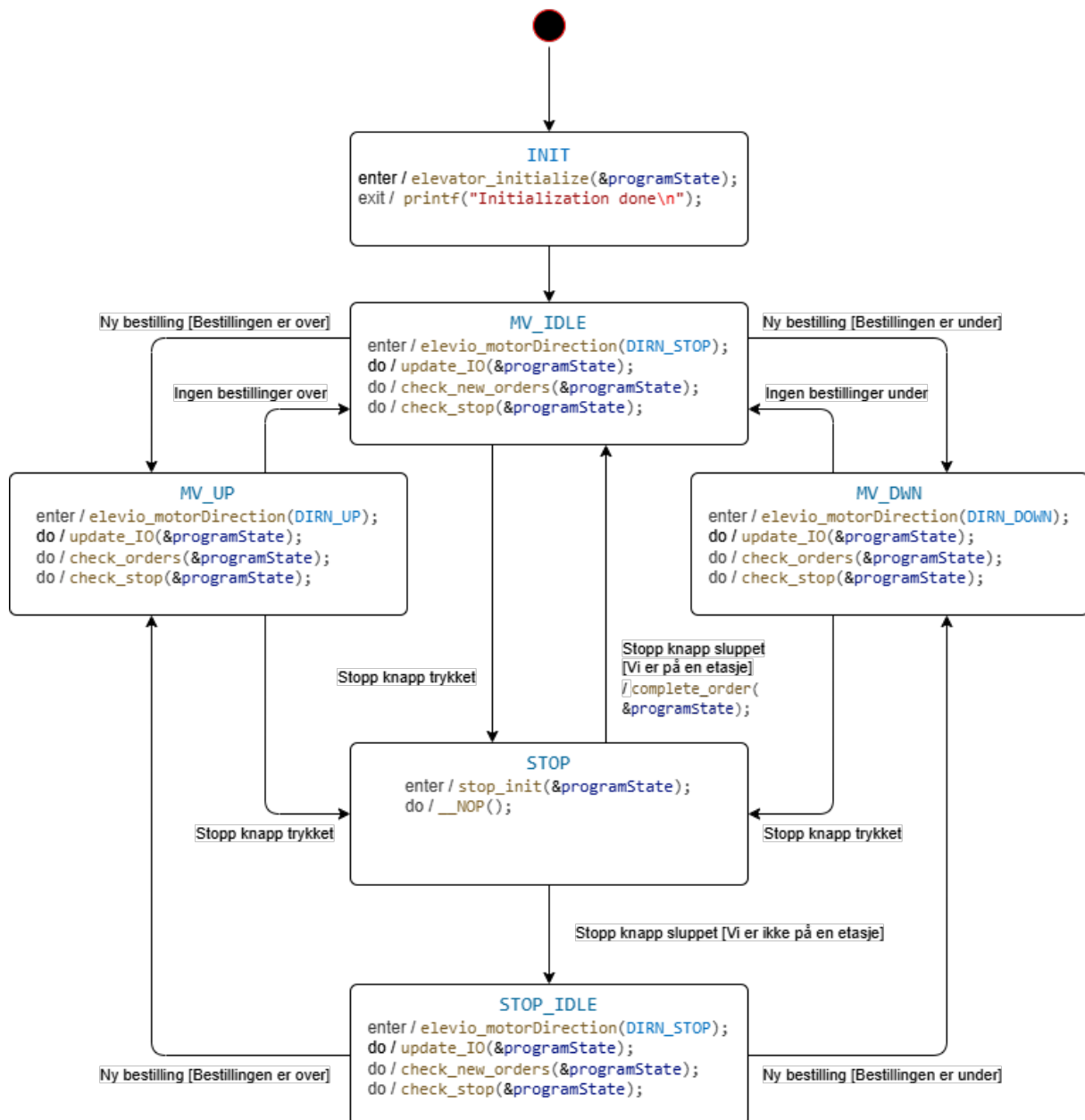
Fellestrekk

Alle tilstander utenom stopp og init kaller Utskrift 6, som oppdaterer indikator lys og ordre basert på inputs og sensorer. Og Utskrift 7 som endrer tilstand til Stopp dersom stoppknappen blir trykket.

Heisens tilstand beskrives med enumen Utskrift 2 og structen Utskrift 3

Initialiserings tilstand

Når programmet startes så sjekker heisen først etter gyldig etasje. Her tar den ingen inputs. Dersom den starter i en gyldig etasje går den umiddelbart til idle state. Hvis ikke så begynner heisen å bevege seg nedover intill den treffer en gyldig etasje. Initialiseringen kan videre beskrives med Figur 5



Figur 3: Fullført tilstandsdiagram

Idle tilstand

I idle tilstand vil programmet lese ordre listen intill den finner en ny ordre. Dersom ordren er i etasjen den er i åpner den bare døren. Ellers vil den endre tilstanden sin til retningen ordren kom fra.

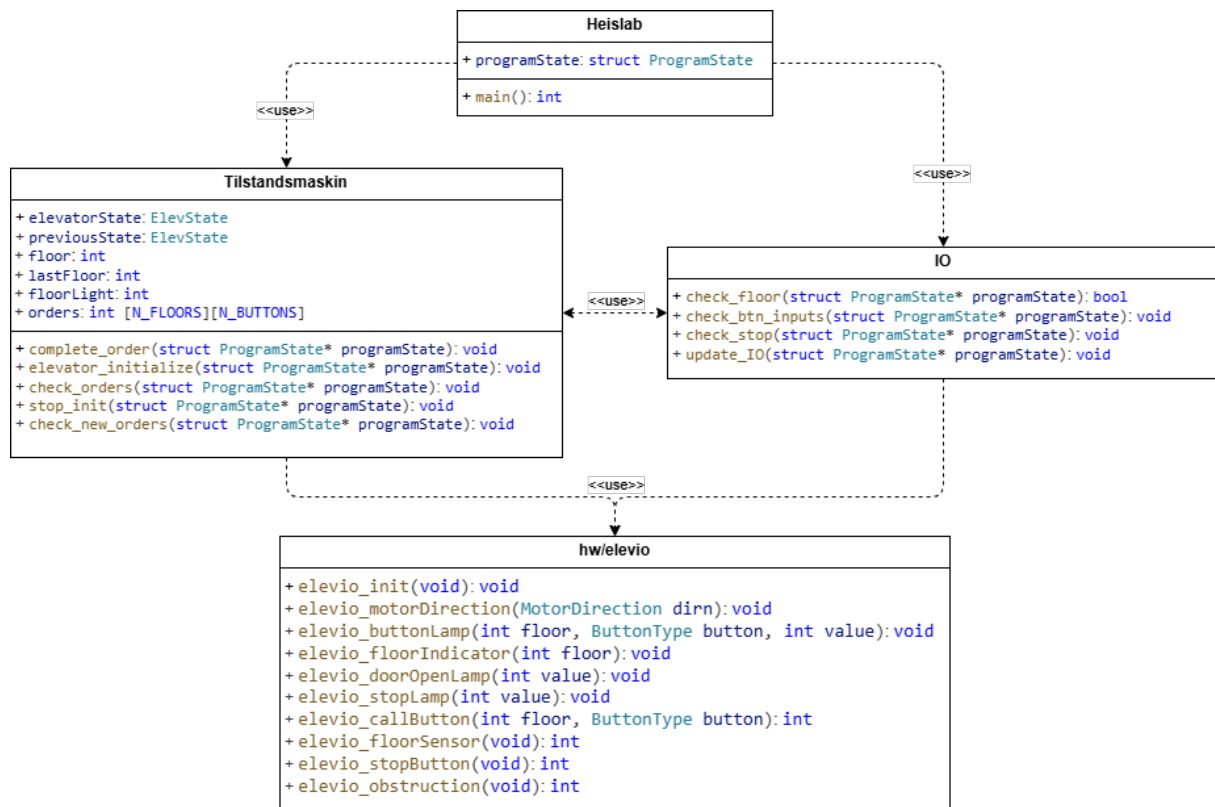
Rettnings tilstand

Dersom heisen er i en rettnings tilstand, altså opp eller ned, så vil motoren være i den korresponderende retningen. Dersom den er i en gyldig etasje vil den sjekke etter ordre. Først sjekker den alle ordre i retningen den går i. Og dersom det er en i etasjen den er i

kaller den Utskrift 4 og deretter Utskrift 5 som sjekker etter flere ordre i bevegelsesretning og endrer tilstand til idle dersom det ikke er noen.

Stopp tilstand

Stopp tilstanden begynner med å slette alle ordre og skru av indikatorlys tilhørende knapper og stopper motoren. En while løkke kjører så lenge stopp knappen er holdt og hindrer andre inputs. Når stopp knappen slippes vil den først sjekke om den er i en gyldig etasje, og dersom den er det kaller den Utskrift 4. Hvis ikke vil tilstanden endres til stopp idle tilstanden.



Figur 4: Klasse diagram

Stopp idle tilstand

Denne tilstanden er et spesialtilfelle av idle tilstanden. Den fungerer på mange måter likt, med unntak av at den avhenger av tilstanden den var i før stopp knappen ble trykket på, for å bestemme retningen den går når den får en ordre. Altså heisen vil alltid vite forrige etasjen den var i, men dersom den er mellom to etasjer så veit for eksempel at den er mellom 2. og 1. etasje dersom forrige etasje var 2 og tilstanden dens er ned. Denne informasjonen bruker den til å fungere virtuelt på samme måte som idle tilstanden (Forrige tilstandsvariabelen vil ikke oppdateres dersom den er i stopp idle når stopp knappen trykkes).

Kodesnutter

```

typedef enum {
    MV_DWN,
    MV_IDLE,
    MV_UP,
    STOP,
    STOP_IDLE,
    INIT} ElevState;
  
```

Utskrift 2: Tilstands enum

```

struct ProgramState{
    ElevState elevatorState;
    ElevState previousState;
    int floor;
    int lastFloor;
    int floorLight;
    int orders[N_FLOORS][N_BUTTONS];
};
  
```

Utskrift 3: En struct som holder på alle variabler som beskriver heisens tilstand

```

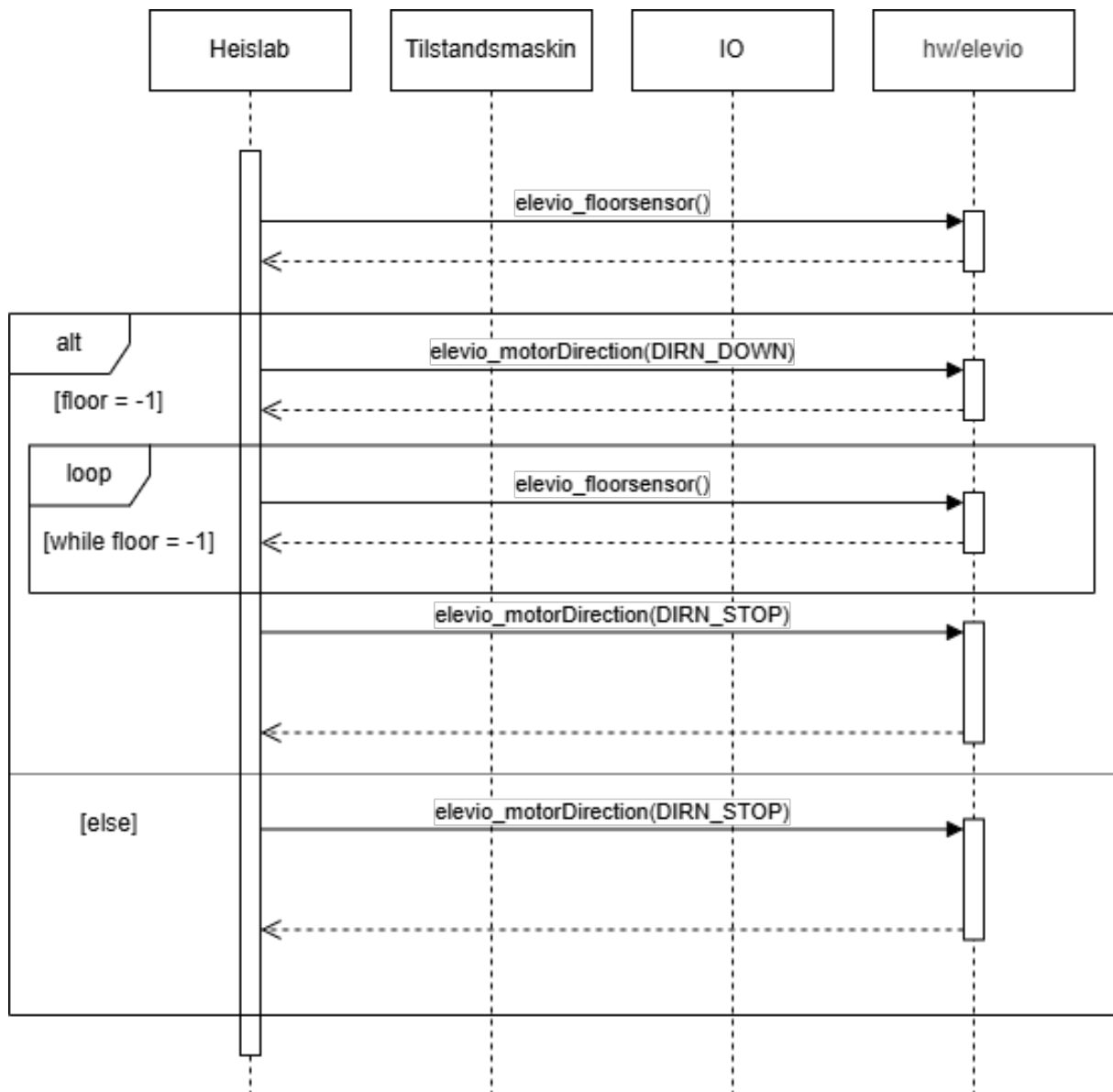
void complete_order(
    struct ProgramState* programState
);
  
```

Utskrift 4: Stopper motoren, åpner døren og fullfører ordre i nåværende etasje

```

void check_orders(
    struct ProgramState* programState
);
  
```

Utskrift 5: Leser gjennom ordre og endrer tilstand til idle dersom det ikke er flere ordre i bevegelsesretning



Figur 5: Sekvensdiagram for initialiseringen

```

void update_IO(
    struct ProgramState* programState
);

```

Utskrift 6: Oppdaterer lys og ordre utifra inputs og sensorer

```

void check_stop(
    struct ProgramState* programState
);

```

Utskrift 7: Umiddelbart endrer tilstand til stopp dersom stoppknappen blir trykket på

Refleksjon

I begynnelsen så var det litt vanskelig å komme i gang, ettersom prosessen enda var litt ukjent. Derfor begynte vi med det kjente, altså å bare teste litt å se hva vi kommer på med. Senere med denne kunnskapen så begynte vi å abstrahere ifølge v-modellen og UML og vi kom frem til at det ga oss et bedre overblikk for programmet slik at vi kunne lage et mer ordentlig og oversiktlig program.